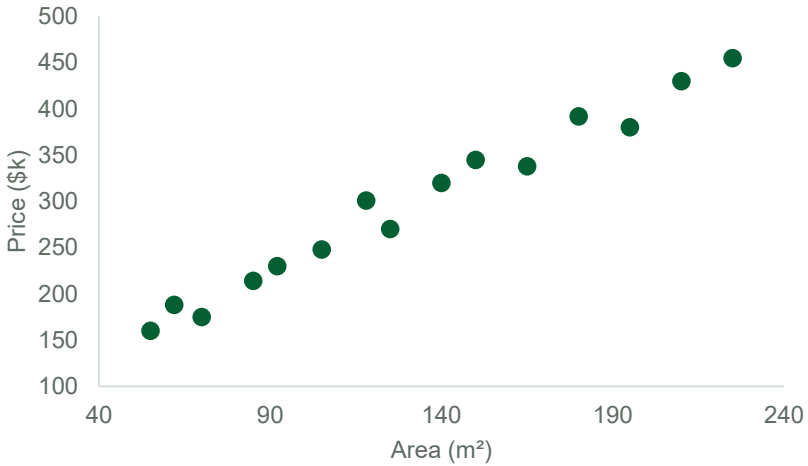
Supervised Learning: Regression

What is regression?

Regression is supervised learning where the target is a number: either continuous (a price, a temperature) or a count (the number of emergency visits).



Numeric targets

- **Continuous:** price, temperature, blood glucose, days in hospital.
- **Counts:** emergency visits per month, insurance claims per year.
- **Goal:** predictions close to the true values on new data.
- **Judged by** how far off a prediction is, not whether it is right or wrong.

Running example: predicting house prices (in $k) from what we know about each house.

Features and target

Features (x) are the inputs we know for each example. The target (y) is the number we want to predict, here the sale price.

Area (m²)	Bedrooms	Location	Age (yr)	Price ($k)
85	2	Suburb	12	214
140	3	City	5	395
62	1	City	30	188
210	4	Rural	18	306
118	3	Suburb	2	?
Features x (inputs)				Target y

1 Observation

One row: a single house.

2 Feature (x)

An input known at prediction time: numbers or categories.

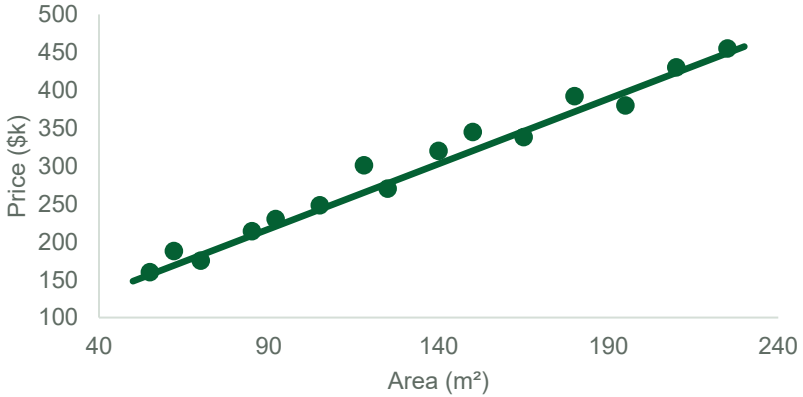
3 Target (y)

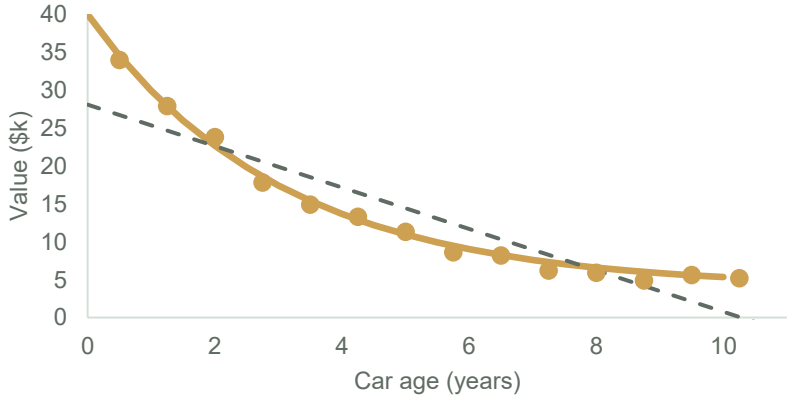
The number to predict; known only for houses already sold.

The last row: a new house. The model's job is to fill in the "?" from its features.

Patterns in data: straight lines and curves

In a linear (straight-line) pattern the target changes by the same amount for every one-unit change in a feature. In a non-linear (curved) pattern that amount changes.





Linear: each extra m² adds about $1,700.

Non-linear: fast drop early, then levels off.

Check first: plot the target against each feature. If the dots bend, a straight line will underfit; Part 3 shows models that can follow curves.

How regression models learn?

Key question

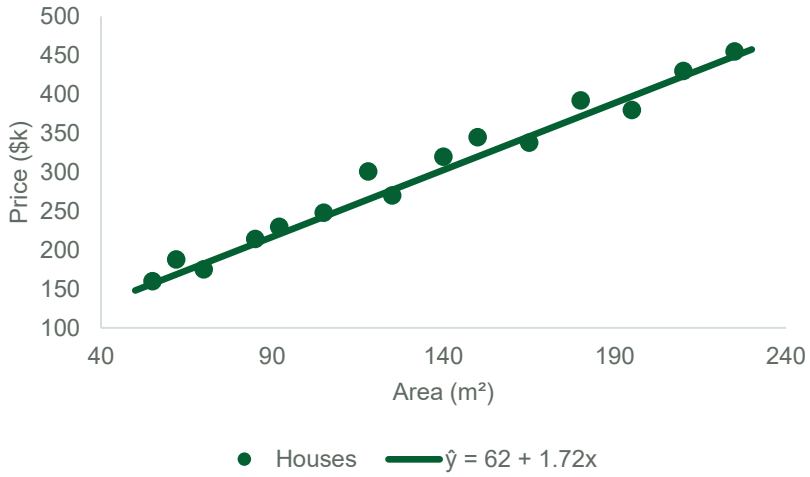
How does a model make predictions and improve them?

1 Model	2 Prediction	3 Residual	4 Loss	5 Learning
Choose a formula	Predict each observation	Compare with the actual value	Combine all residuals	Adjust parameters to lower the loss
$\hat{y} = b_0 + b_1 x$	$\hat{y}_i$	$e_i = y_i - \hat{y}_i$	$MSE = \frac{1}{n} \Sigma (y_i - \hat{y}_i)^2$	**minimise MSE**

Steps 2 to 5 repeat: predict, compare, measure, adjust, and predict again, until the loss stops falling.

Step 1: The model

A model is a formula with adjustable numbers called parameters. The simplest regression model is a straight line, $\hat{y} = b_0 + b_1 x$.



Reading $\hat{y} = b_0 + b_1 x$

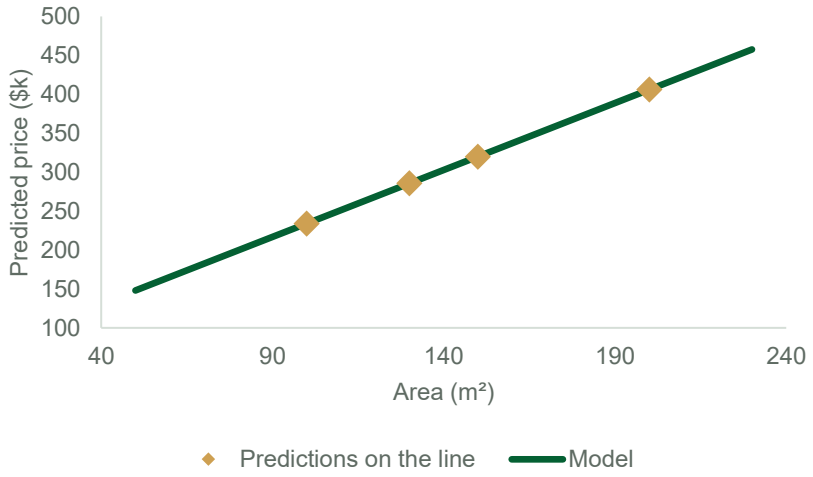
- **x:** the feature (area in m²).
- **ŷ:** the predicted price ("y-hat").
- **b_0 (intercept):** where the line crosses the y-axis; here 62.
- **b_1 (slope):** price change per extra m²; here 1.72 ($1,720).
- **Parameters:** b_0 and b_1 are what the model learns.

Simple linear regression: one feature, one straight line. Other models in Part 3 use different formulas, but the same five steps.

Step 2: The prediction

A prediction ŷ is the value the model outputs for an observation when its features are put into the formula.

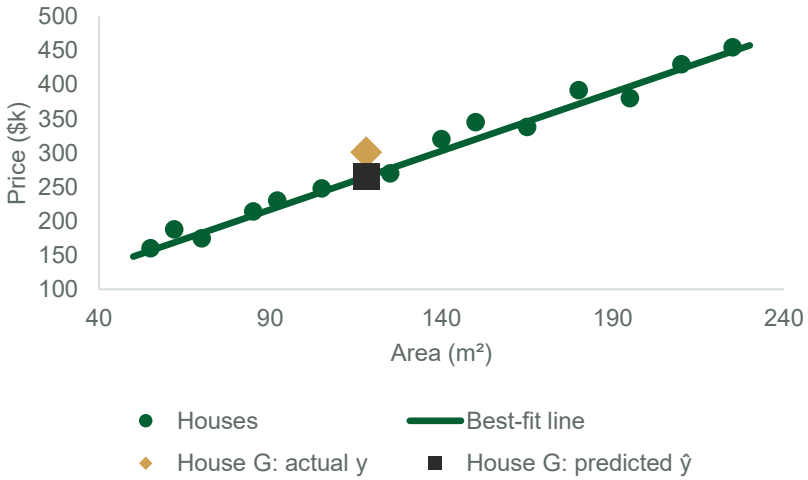
House	Area x	ŷ = 62 + 1.72x
A	100 m²	62 + 172 = $234k
B	150 m²	62 + 258 = $320k
C	200 m²	62 + 344 = $406k
New house	130 m²	**62 + 224 = $286k**



Every prediction lies on the line. The model gives a ŷ for any house, including ones it has never seen.

Step 3: The residual

A residual is the difference between the actual and predicted value for one observation: $e_i = y_i - \hat{y}_i$. Positive means the model predicted too low.



House G

Actual price y	**$301k**
Predicted price ŷ	**$265k**
Residual e = y − ŷ	**+$36k**

Positive: predicted too low. Negative: predicted too high.

Every house has a residual. Step 4 combines them into one number.

Step 4: The loss (objective)

A loss function, or objective, combines all the residuals into one measure of model error. The most common for regression is the mean squared error (MSE).

House	Actual y	Predicted ŷ	Residual	Squared
A	$300k	$280k	+20	400
B	$250k	$260k	−10	100
C	$420k	$390k	+30	900
D	$180k	$190k	−10	100
MSE				**375**

Mean squared error

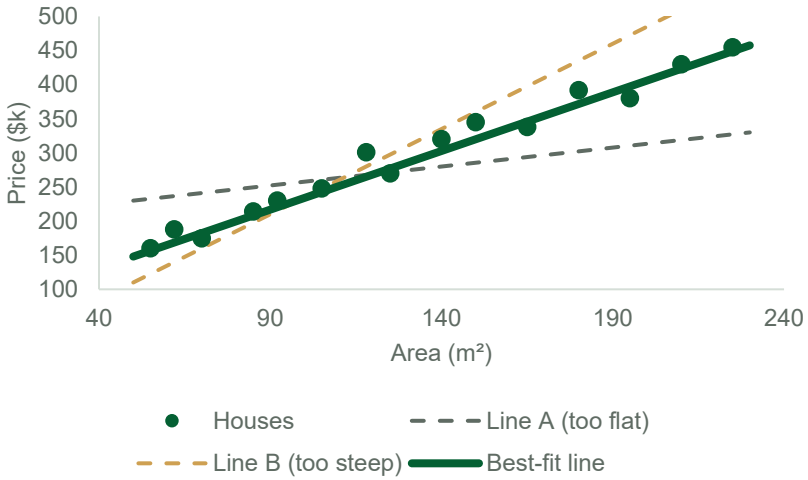
$$\text{MSE} = \frac{1}{n} \Sigma (y_i - \hat{y}_i)^2$$

- **Square:** no cancelling; big misses count more.
- **Average:** divide by the number of houses, n.
- **Alternative:** MAE averages $|y_i - \hat{y}_i|$ without squaring.

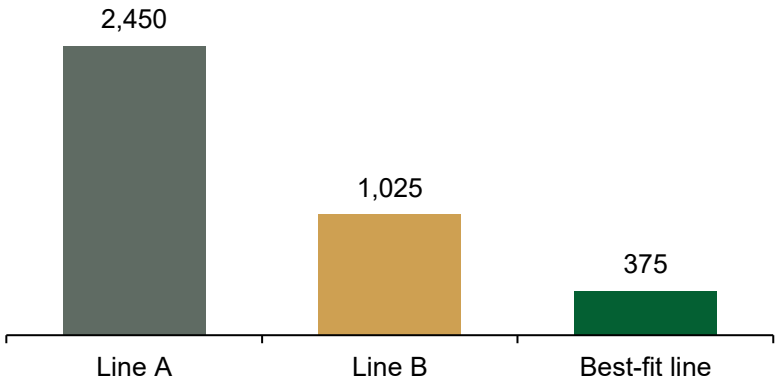
One number for the whole model: lower MSE means the predictions are closer to the actual prices overall.

Step 5: learning (fitting)

Learning, or fitting, means finding the parameter values that minimise the objective. For a straight line with MSE this is called ordinary least squares.



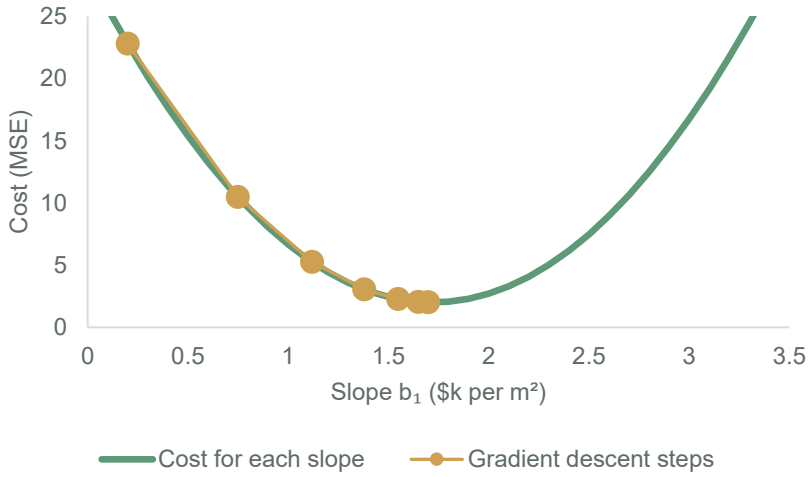
MSE of each line: lowest wins



How to search: a formula finds the best straight line directly; most other models use gradient descent (next).

Gradient descent

Gradient descent is a step-by-step method that changes the parameters a little at a time in the direction that lowers the loss, until the loss stops falling.



1. **Start:** Pick any starting slope and intercept.
2. **Measure:** Compute the cost (MSE) of that line.
3. **Step downhill:** Change the parameters a little in the direction that lowers the cost.
4. **Repeat:** Stop when the cost no longer falls.

Learning rate: the size of each step. Too big overshoots the bottom; too small takes forever.

Types of regression models

Key question

What different approaches can learn the relationship?

Multiple linear	Polynomial	Decision tree	Random forest	Gradient boosting	k-NN, SVR, Poisson

Multiple linear regression

Multiple linear regression extends the best-fit line to several features: $\hat{y} = b_0 + b_1x_1 + b_2x_2 + ... + b_kx_k$, with one coefficient per feature.

L Linearity

Each feature relates to the target roughly in a straight line.

I Independence

Rows do not influence each other (careful with repeated measures).

N Normal residuals

Errors are roughly bell-shaped; matters mainly for intervals.

E Equal variance

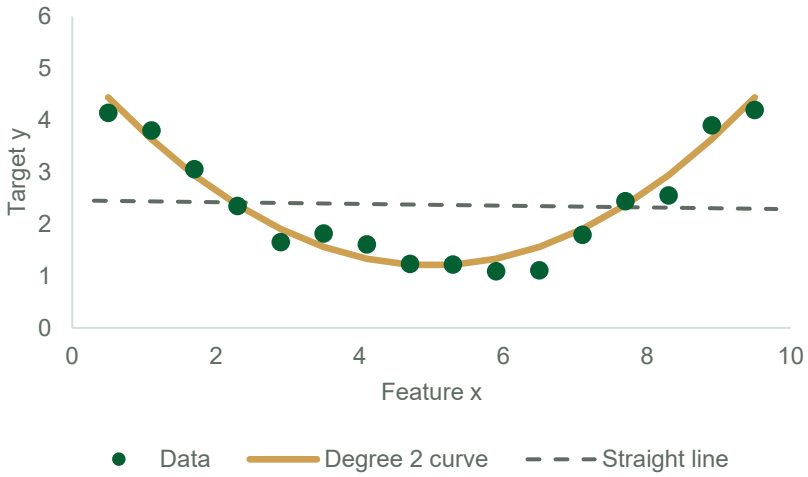
Errors have a similar spread everywhere (homoscedasticity).

Reading a coefficient: the change in the target for one more unit of that feature, holding all other features fixed.

Multicollinearity: near-duplicate features (area in ft² and m²) make coefficients unstable. Drop one, or use ridge.

Polynomial regression

Polynomial regression adds powers of a feature (x^2, x^3, ...) as new features, so a linear model can fit a curve.



How it works

- Add x^2, x^3, ... as extra features.
- Fit an ordinary linear model on them.
- The result is a smooth curve in x.
- Higher degree = more flexible curve.

Key hyperparameters

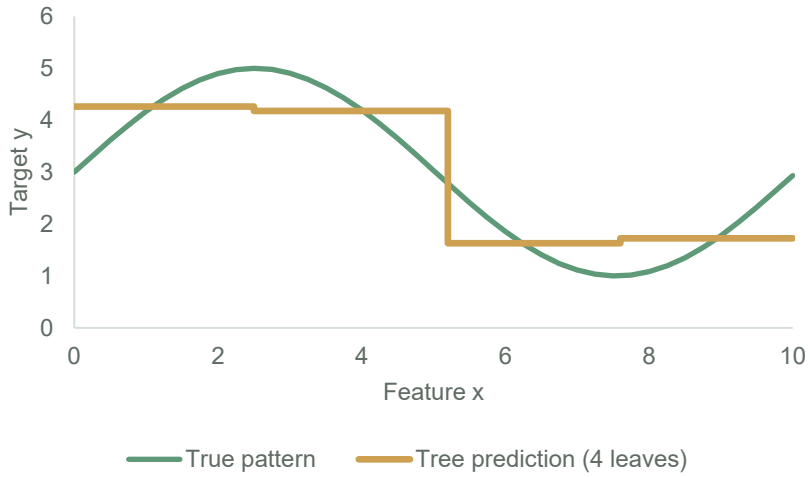
- Degree: choose by cross-validation
- Scale features before raising to powers

Use when: the pattern is a smooth curve in one or two features.

Avoid when: there are many features: the number of terms explodes.

Decision tree regression

A regression tree splits the feature space into regions with yes/no questions and predicts the average target of the training rows in each region.



How it works

- Split the data with yes/no questions on the features.
- Each split reduces the squared error most.
- A leaf predicts the average target of its rows.
- Prediction looks like a staircase.

Key hyperparameters

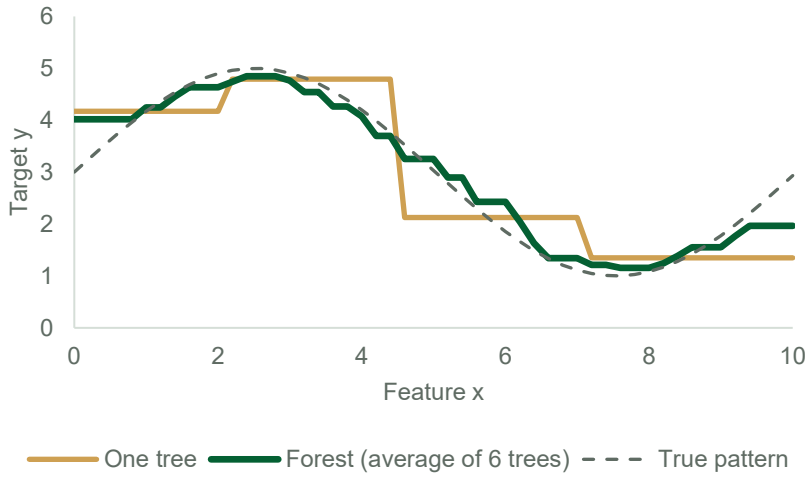
- Max depth; min samples per leaf
- More depth = more steps = more overfitting

Use when: you need readable rules and a non-linear fit.

Avoid when: used alone (unstable) or when extrapolation is needed.

Random forest regression

A random forest regressor averages the predictions of many regression trees, each trained on a bootstrap sample with random features at each split.



How it works

- Grow many deep trees on bootstrap samples.
- Use random features at each split.
- Average the trees' predictions.
- Smoother and more stable than one tree.

Key hyperparameters

- Number of trees; max features per split
- Min samples per leaf

Use when: you want a strong default on tables with little tuning.

Avoid when: you must extrapolate beyond the training range.

Gradient boosting regression

Gradient boosting builds shallow trees one after another, each fitted to the residuals left by the trees before it, and adds them with a small learning rate.

Step	Prediction	Residual
Start: mean	$250k	+$50k
+ 0.5 × tree 1 (+$40k)	$270k	+$30k
+ 0.5 × tree 2 (+$30k)	$285k	+$15k
+ 0.5 × tree 3 (+$14k)	$292k	+$8k
... after many trees	≈ $300k	≈ $0

One house, true price $300k, learning rate 0.5. Each tree is trained on what is still wrong.

How it works

- Start from the mean of the target.
- Fit a shallow tree to the current residuals.
- Add a fraction of it (learning rate).
- Repeat; stop when validation error stops improving.

Key hyperparameters

- Number of trees + early stopping; learning rate
- Tree depth (3-8); row and column subsampling

Use when: you want top accuracy on tabular data.

Avoid when: you need extrapolation or a simple explanation.

Other regression models

k-NN, SVR and Poisson regression are three more model families: one averages similar examples, one fits a tube around a function, one models counts.

1 k-NN regression

Averages the targets of the k most similar training examples.

Use for: Small data with local patterns. Scale features first.

2 Support vector regression

Fits a function with a tube of width ε; only points outside the tube shape it.

Use for: Small to medium data with a smooth pattern.

3 Poisson regression

A linear model for counts that predicts through a log link, so it is never negative.

Use for: Counts: visits, claims, defects per month.

Also worth knowing: neural networks (for images, text, audio) and tabular foundation models (small tables, no training).

Generalisation and controlling complexity

Key question

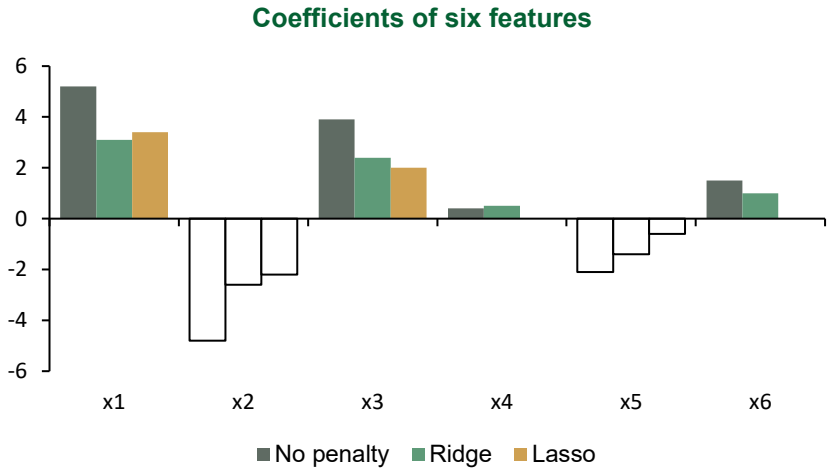
How do we avoid overfitting?

Linear models: ridge, lasso, elastic net

Tree models: limiting growth

Regularisation for linear models

Regularisation limits a model's complexity to reduce overfitting. For linear models it adds a penalty on large coefficients to the cost: cost = MSE + α × penalty.



Method	Penalty	Effect
Ridge	Sum of squared coefficients (L2)	Shrinks all; none become zero
Lasso	Sum of absolute coefficients (L1)	Sets some to zero: picks features
Elastic net	Mix of L1 and L2	Shrinks and selects

In practice: standardise features, choose α by cross-validation, and start with ridge.

Regularisation for tree models

Tree models are regularised by limiting how much they can grow: tree depth, leaf size, number of trees and learning rate.



Knob	Effect
Max depth	Fewer questions in a row
Min samples per leaf	Bigger, smoother leaves
Features per split (forest)	More diverse trees
Learning rate (boosting)	Smaller, safer steps
Early stopping (boosting)	Stop when validation stops improving
Subsampling (boosting)	Each tree sees part of the data

Same idea as ridge and lasso: limit complexity, and choose the setting on validation data.

Part 5: Evaluating regression models

Key question

How do we know whether the model works?

MAE	MSE	RMSE	R^2	Residual plots	Choosing a metric

Mean absolute error (MAE)

MAE is the average size of the errors, ignoring their sign: MAE = (1/n) Σ $|y_i - \hat{y}_i|$. It is in the same units as the target.

House	Actual y	Predicted ŷ	Error y − ŷ	|Error|
1	$250k	$240k	+10	10
2	$300k	$320k	−20	20
3	$180k	$200k	−20	20
4	$420k	$380k	+40	40
5	$310k	$300k	+10	10
Sum				**100**

$$\text{MAE} = \frac{1}{n} \Sigma |y_i - \hat{y}_i|$$

100 / 5 = $20k

- **Read it as:** on average, predictions are $20k off.
- **Units:** same as the target ($k).
- **Big errors:** count in proportion; outliers do not dominate.

Use MAE when: you want a typical error that anyone can understand, in the target's own units.

Mean squared error (MSE)

MSE is the average of the squared errors: MSE = $(1/n)\ \Sigma\ (y_i - \hat{y}_i)^2$. Squaring makes large errors count much more than small ones.

House	Actual y	Predicted ŷ	Error y − ŷ	Error²
1	$250k	$240k	+10	100
2	$300k	$320k	−20	400
3	$180k	$200k	−20	400
4	$420k	$380k	+40	1,600
5	$310k	$300k	+10	100
Sum				**2,600**

$$\text{MSE} = \tfrac{1}{n}\ \Sigma\ (y_i - \hat{y}_i)^2$$

$$2{,}600 / 5 = 520$$

- **Big errors dominate:** house 4 alone gives 62% of the total.
- **Units:** squared ($k²), so hard to interpret.
- **Familiar:** the same formula as the loss in Step 4.

Use MSE when: large errors are especially costly. Report it as RMSE so people can read it.

Root mean squared error (RMSE)

RMSE is the square root of MSE: RMSE = √MSE. It keeps MSE's extra weight on large errors but is back in the target's units.

RMSE = √MSE

√520 = $22.8k

- **Read it as:** a typical error of about $23k.
- **Units:** back in the target's units ($k).
- **Big errors:** still count more, as in MSE.

Same MAE, different RMSE

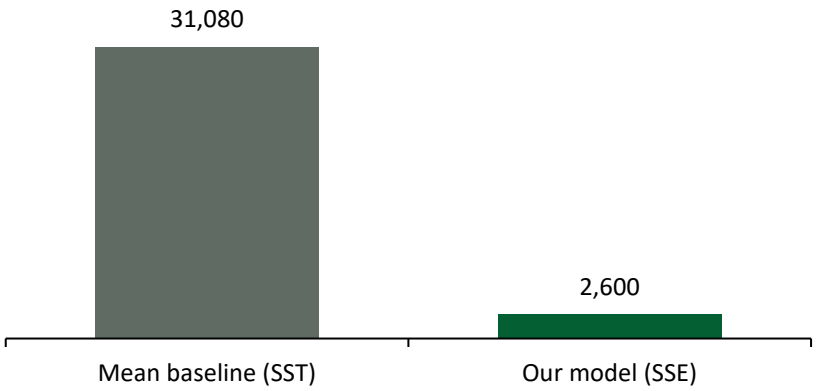
	Model A	Model B
Errors on 5 houses	20, 20, 20, 20, 20	0, 0, 0, 0, 100
MAE	20	20
RMSE	20	**44.7**

RMSE reveals model B's single huge miss; MAE hides it.

RMSE vs MAE: RMSE is always at least MAE. A big gap means a few large errors dominate.

R^2 and adjusted R^2

R^2 is the share of the variation in the target that the model explains, compared with always predicting the mean. Adjusted R^2 also penalises extra features.



$R^2 = 1 - 2{,}600 / 31{,}080 = 0.92$

How to read R^2

- **1:** perfect predictions.
- **0:** no better than always predicting the mean.
- **Below 0:** worse than the mean (possible on test data).
- **High R^2 ≠ small errors:** report it with MAE or RMSE.
- **Adjusted R^2 (optional):** penalises extra features; fairer when feature counts differ.

In words: the model removes 92% of the error we would make by always predicting the average price.

What is a residual plot?

A residual plot is a scatter plot of each observation's residual ($y - \hat{y}$) against its predicted value $\hat{y}$, with a line at zero.



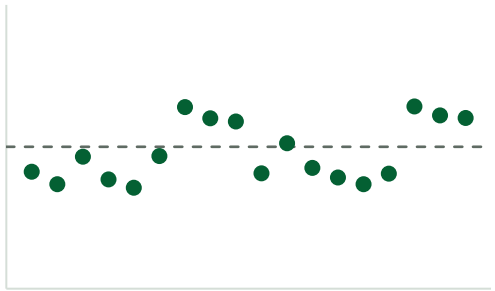
1. **Predict:** Get $\hat{y}$ for each test observation.
2. **Residual:** Compute $y - \hat{y}$ for each one.
3. **Plot:** Residual on the y-axis, $\hat{y}$ on the x-axis.
4. **Zero line:** Above: under-predicted. Below: over-predicted.

Why use it: a metric gives one number; the plot shows where and how the model is wrong.

Reading residual plots

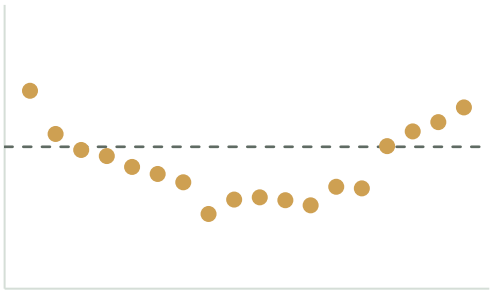
A pattern in a residual plot means the model is missing something systematic. A healthy plot shows only random scatter around zero.

Healthy: random cloud



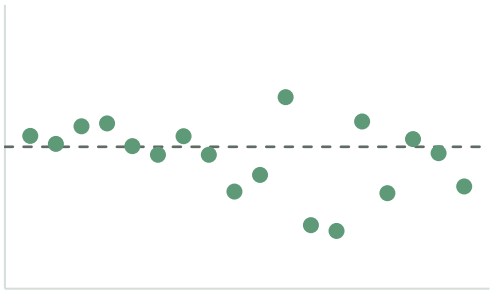
Healthy: nothing left to learn.

Curve: missing non-linearity



Curve: add curvature or a non-linear model.

Funnel: spread grows



Funnel: try modelling log(y).

Outliers: check whether extreme residuals are data errors or real cases before removing anything.

Choosing the right evaluation metric

The right metric is the one that matches the real cost of errors for the people who will use the predictions.

If this matters most...	Use	Because
Errors should count in proportion; some outliers	MAE	Every $1 of error counts the same
Big errors are especially costly	RMSE	Squaring makes big errors count more
Explaining the error to non-experts	MAE or RMSE	Both are in the target's units
Error relative to the size of the value	MAPE	Percent error; avoid near zero
Comparing models on the same data	R^2 (plus MAE / RMSE)	Share of variation explained

Pitfalls to avoid: metrics on training data; R^2 reported alone; leakage (features known only after the event, or preprocessing before splitting); and predicting far outside the training range.